

2D Systolic Array Innovation Ideas

Edge-AI Hackathon 2026 — Innovation Exploration Report

Purpose: Consolidate the innovation directions discussed for the submitted 2D systolic-array project so they can be reviewed offline before selecting one architecture improvement.

Current position

The team has already submitted an abstract for a scalable 2D systolic array. The next objective is not to replace the assigned problem, but to identify one clear, measurable architectural improvement over a conventional implementation.

Important: These are candidate innovation directions, not yet a finalized architecture. The intended process is: investigate → compare → select one → verify feasibility → ask the organizer whether the submitted architecture can be refined.

1. Workload-Adaptive / Utilization-Aware Systolic Array

Current favorite. The physical FPGA array remains fixed (for example, 8×8), but the architecture adapts the active computation to the actual workload dimensions.

Example: An 8×8 array contains 64 physical PEs. A 5×7 workload contains 35 useful matrix positions. A conventional fixed architecture may have computation that is not useful for the current workload. The proposed direction is to make the array workload-aware and control which PE region performs useful work.

Concept: Input dimensions $\rightarrow$ dimension/controller logic $\rightarrow$ determine active PE region $\rightarrow$ enable the required computation $\rightarrow$ run the systolic pipeline.

Potential measurements: PE/MAC utilization; latency; throughput; dynamic power or activity; FPGA resource usage.

Important engineering caution: Do not automatically equate '35 useful PE positions out of 64' with 54.7% systolic utilization. PE utilization is temporal as well as spatial. A rigorous metric should count useful MAC operations against available MAC opportunities over the execution interval.

2. Irregular-Matrix / Non-Square Workload Optimization

A broader version of the adaptive idea: design the architecture to handle workloads such as 3×7 , 5×13 , 7×11 , or other non-square dimensions efficiently rather than assuming that workloads naturally match the physical array dimensions.

Possible positioning: Dimension-Aware Systolic Array or Workload-Adaptive Systolic Array. The central question is how efficiently arbitrary dimensions can be mapped onto a fixed PE grid.

3. Resource-Aware Systolic Array

Optimize the architecture for FPGA resource consumption while maintaining useful performance. The comparison would focus on LUTs, flip-flops, DSPs, BRAM, and timing.

Target claim: achieve comparable computational performance with lower resource overhead, supported by Vivado synthesis and implementation reports.

4. Power-Aware / Energy-Efficient Systolic Array

Reduce unnecessary switching in PEs that are not contributing to the current workload. Candidate mechanisms include clock enables, operand isolation, data gating, or activity-aware PE enables.

Target claim: same required computation with reduced unnecessary switching and lower estimated/observed power.

5. Adaptive-Precision Systolic Array

Allow different numerical precisions such as INT8, INT4, or INT2 depending on workload. Lower precision can potentially increase computational density and reduce hardware cost, but this introduces additional AI/quantization complexity.

6. Data-Reuse / Memory-Bandwidth Optimization

Optimize how data reaches the PE array using local buffering, BRAM organization, tiling, input reuse, or scheduling. The objective is to increase useful computation per memory access and avoid starving the array.

7. Dynamic / Runtime-Reconfigurable Array Size

Distinguish compile-time parameterization from runtime adaptation. A physical 8×8 array could potentially operate using smaller active regions such as 2×2 , 4×4 , 5×7 , 6×8 , or 8×8 depending on workload.

8. Sparse / Zero-Skipping Systolic Array

Detect zero or otherwise inactive operands and avoid unnecessary MAC operations. This could reduce work and power for sparse workloads, but requires additional control/dataflow mechanisms and a clear definition of sparsity.

9. Latency-Optimized Systolic Array

Investigate the pipeline and scheduling structure to reduce time from input arrival to valid output without sacrificing timing or throughput. This is potentially powerful but technically harder to optimize safely.

10. Hybrid-Dataflow Systolic Array

Explore switching or combining dataflow strategies such as output-stationary, weight-stationary, and input-stationary approaches according to workload. This is a high-complexity direction and is not the recommended starting point.

11. Fault-Tolerant / Reliability-Aware Systolic Array

Introduce mechanisms such as error detection, redundant computation, faulty-PE bypass, or recovery. Interesting but significantly more complex and potentially too close to the separate fault-tolerant processor problem statement.

Shortlist for This Project

Direction	Innovation	Difficulty	FPGA fit	Current fit
Workload-Adaptive / PE utilization	★★★★★	3.5/5	★★★★★	★★★★★
Irregular-dimension optimization	★★★★★	4/5	★★★★★	★★★★★
Resource-aware	★★★★■	3.5/5	★★★★★	★★★★★
Power-aware	★★★★■	4/5	★★★★★	★★★★■
Adaptive precision	★★★★★	4.5/5	★★★★■	★★★██
Memory/data reuse	★★★★■	4/5	★★★★★	★★★★■
Runtime reconfiguration	★★★★■	4/5	★★★★■	★★★★■
Sparse/zero-skipping	★★★★★	4.5/5	★★★★■	★★★██
Latency optimization	★★★★■	4.5/5	★★★★★	★★★██
Hybrid dataflow	★★★★★	5/5	★★★★■	★★███
Fault tolerance	★★★★■	5/5	★★★██	★★███

Current top three: (1) Workload-Adaptive + PE utilization, (2) Resource-aware + workload-adaptive, (3) Workload-adaptive + power-aware.

Recommended Decision Process

Do not combine ideas yet. First investigate the simplest candidate—the workload-adaptive direction—and determine whether it produces a real, measurable advantage over a conventional fixed array.

Step	Action	Purpose
1	Research existing limitations	Understand how conventional systolic arrays behave with workloads that do not match the pattern.
2	Define the baseline	Specify the fixed array, workload dimensions, timing, and utilization metric.
3	Design the proposed mechanism	Decide exactly how the active PE region is identified and controlled.
4	Benchmark	Compare baseline and proposed designs using utilization, latency, throughput, power/activity.
5	Select the innovation	Only retain the idea if the improvement is technically meaningful and demonstrable.
6	Ask the organizer	Then ask whether the submitted architecture/abstract can be refined or updated.

Important principle: Do not claim innovation merely because a feature sounds new. The intended hackathon story is: conventional architecture → specific limitation → architectural modification → measurable improvement. The final result should be demonstrated with actual simulation and FPGA implementation data.

The strongest presentation is likely to be a baseline-versus-proposed comparison where the numbers—PE utilization, latency, throughput, power/activity, and resource usage—demonstrate the benefit.

Shared Project Context with Antigravity

A separate idea discussed for keeping ChatGPT and Antigravity aligned is to maintain a shared project knowledge base rather than trying to transfer the entire conversation. Suggested files include: MASTER_CONTEXT.md, SUBMITTED_ABSTRACT.md, PROBLEM_STATEMENT.md, DESIGN_DECISIONS.md, RESEARCH_NOTES.md, INNOVATION_CANDIDATES.md, and ROADMAP.md.

A later MCP server could expose functions such as `get_project_context()`, `get_submitted_abstract()`, `get_current_architecture()`, `get_design_decisions()`, `get_innovation_candidates()`, and `search_project_knowledge()`. The project files should remain the source of truth.

For now: do not build the MCP bridge yet. First finalize the architectural innovation. Once the innovation is selected, create the shared project context so Antigravity can implement the same agreed architecture without losing the project history.

End of report